



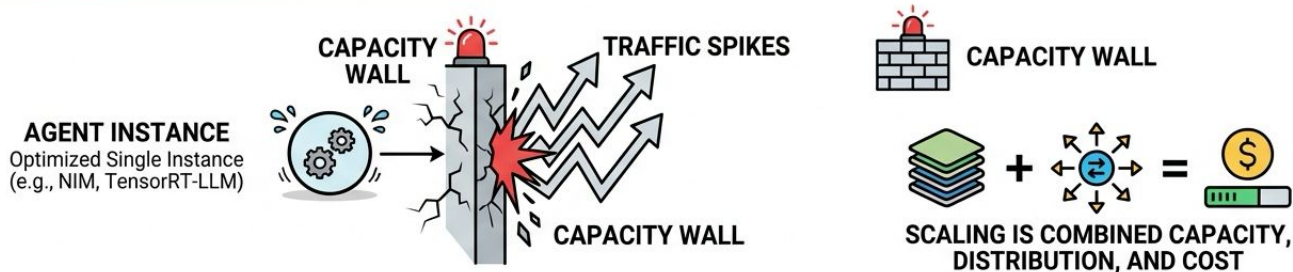
U.S. General Services
Administration

Chapter 4.7: Scaling Strategies

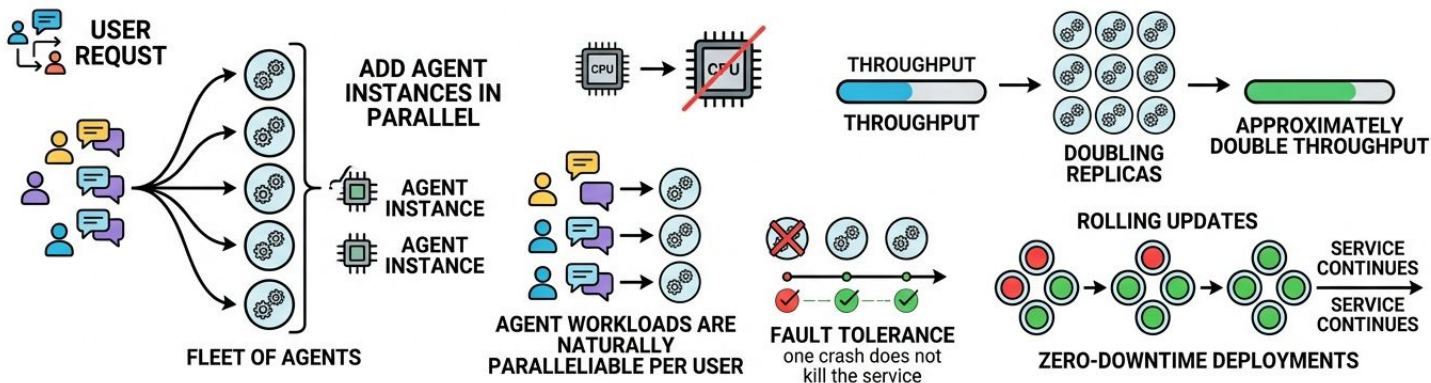
US General Service Administration
AI Community of Practice - March 2026

Horizontal Scaling

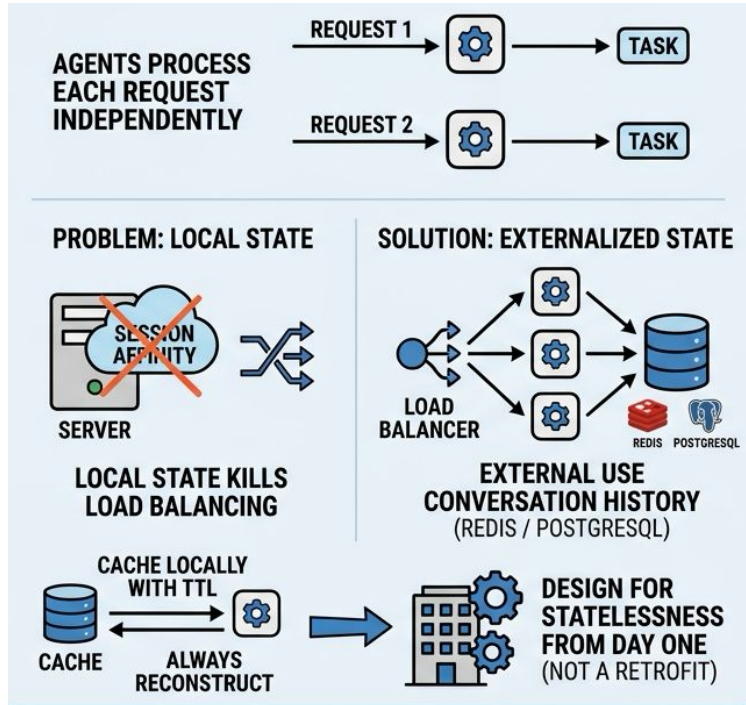
ONE AGENT IS NEVER ENOUGH



HORIZONTAL SCALING -- BUILDING AGENT FLEETS

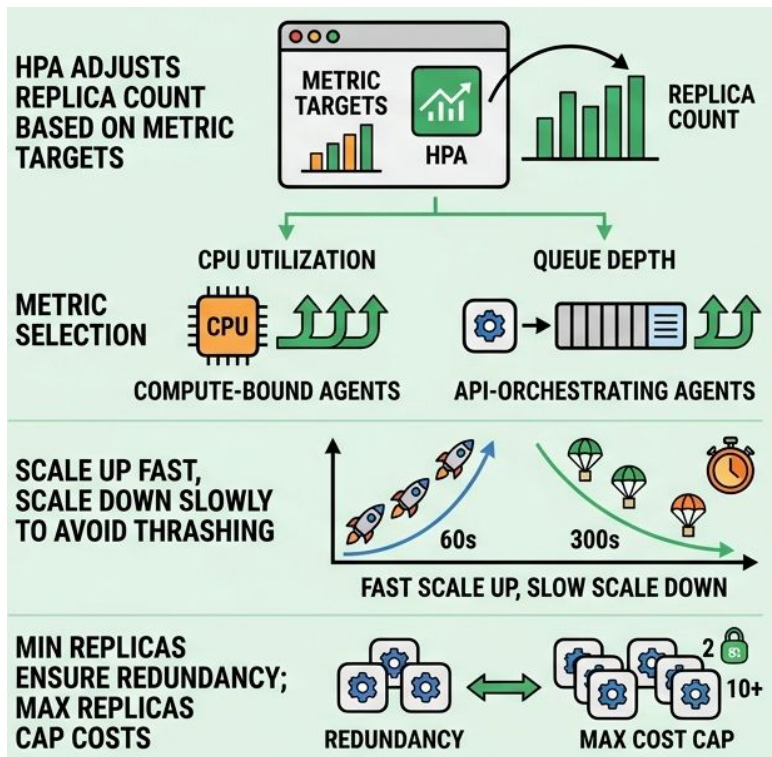


The Stateless Design Imperative



- Agents must process each request independently
- Local state creates session affinity that kills load balancing
- Externalize conversation history to Redis or PostgreSQL
- Cache locally with TTL, but always be able to reconstruct
- Design for statelessness from day one, not as a retrofit

Auto-Scaling -- Let the System Decide



- HPA adjusts replica count based on metric targets
- CPU utilization works for compute-bound agents
- Queue depth better for API-orchestrating agents
- Scale up fast, scale down slowly to avoid thrashing
- Min replicas ensure redundancy; max replicas cap costs

Load Balancing & Batching

LOAD BALANCING ALGORITHMS



ROUND ROBIN
SIMPLE DISTRIBUTION |
BLIND TO ACTUAL WORKLOAD



LEAST CONNECTIONS
ADAPTS TO VARIABLE
REQUEST DURATIONS |
ROUTES TO LEAST ACTIVE SERVERS



WEIGHTED ROUND ROBIN
MATCHES HETEROGENEOUS
HARDWARE |
PROPORTIONAL TO SERVER CAPACITY



IP HASH
PROVIDES SESSION AFFINITY
REDUCED BALANCE

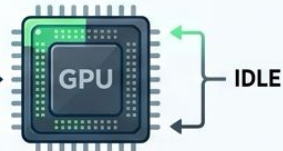


DYNAMIC METRICS-BASED ROUTING
MOST SOPHISTICATED
CONSIDERS REAL-TIME PERFORMANCE METRICS



BATCHING: GPU EFFICIENCY THROUGH GROUPING

SINGLE-REQUEST
PROCESSING
WASTES 80-85%
GPU CAPACITY



STATIC BATCHING



WAITS FOR N REQUESTS
HIGH THROUGHPUT
VARIABLE LATENCY



DYNAMIC BATCHING



ADDS TIMEOUT
LATENCY BOUNDS



CONTINUOUS BATCHING



INSERTS REQUESTS
BETWEEN TOKEN STEPS
MAXIMIZES THROUGHPUT

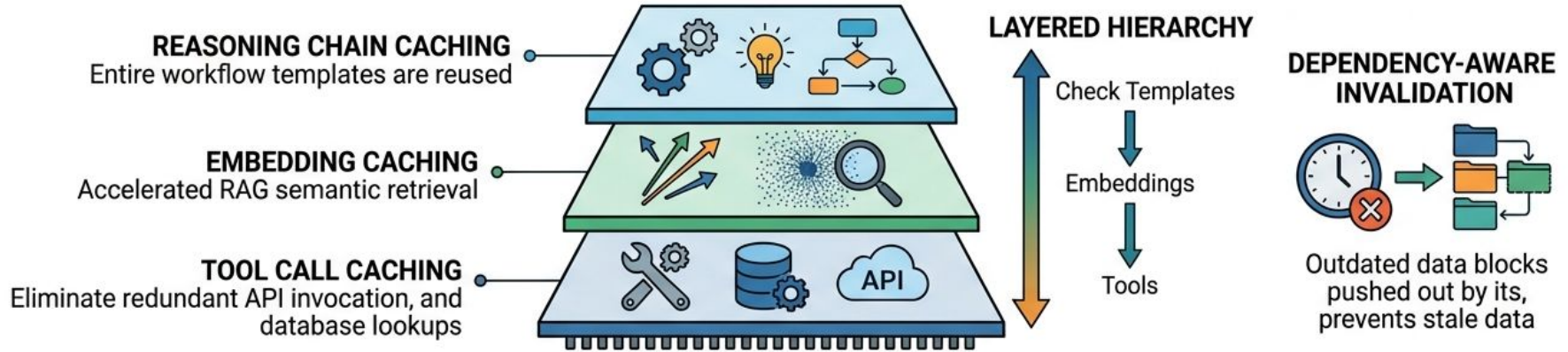


TRITON INERENCE SERVER SUPPORT



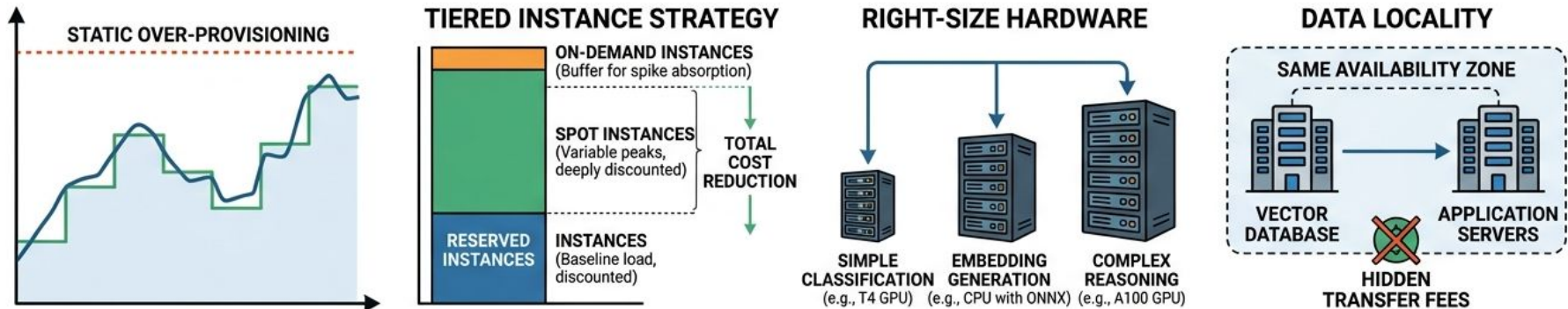
CONFIGURABLE DYNAMIC BATCHING

Multi-Tier Caching Architecture



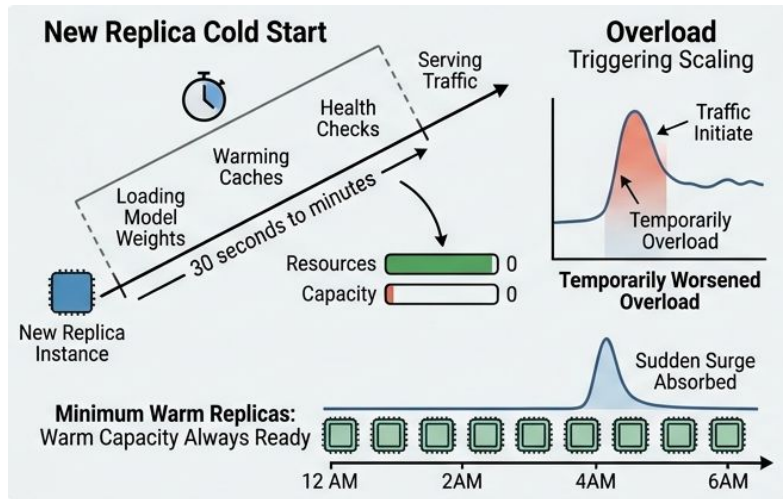
- Tool call caching eliminates redundant API invocations
- Embedding caching accelerates RAG semantic retrieval
- Reasoning chain caching reuses entire workflow templates
- Layered hierarchy: check templates, then embeddings, then tools
- Dependency-aware invalidation prevents serving stale data

Cost Optimization

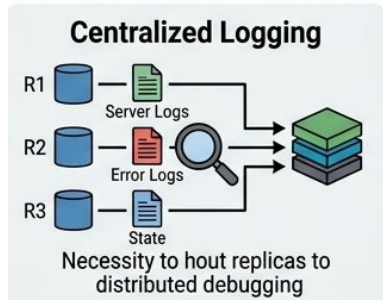


- Elastic scaling replaces static over-provisioning
- Reserved instances for baseline; spot instances for spikes
- Spot plus on-demand buffer achieves ~55% cost reduction
- Right-size hardware: not every task needs an A100
- Data locality eliminates hidden transfer fees

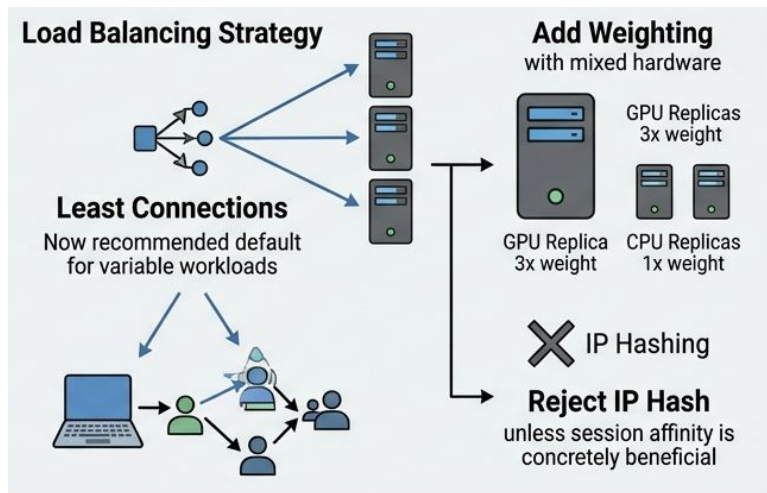
Cold Starts and Operational Complexity



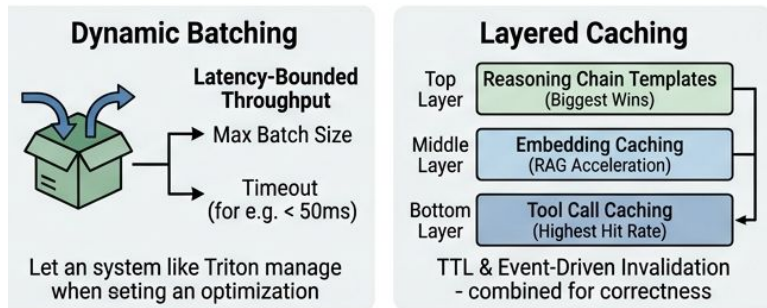
- New replicas take 30 seconds to minutes before serving traffic
- Cold starts worsen the overload that triggered scaling
- Maintain minimum warm replicas even during low traffic
- Centralized logging is mandatory for distributed debugging
- Monitor fleet-wide health, not just individual instances



The Decision Framework

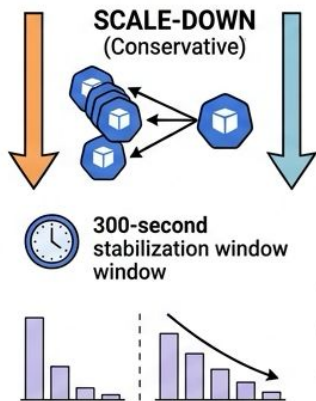
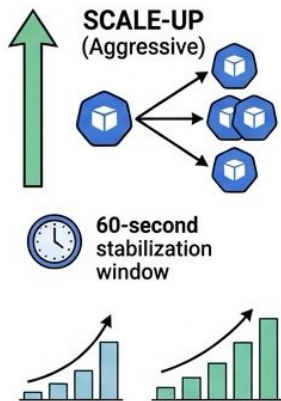
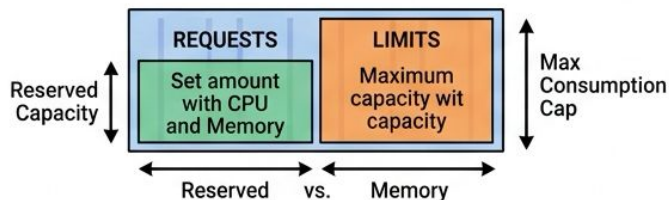


- Start with least connections for variable workloads
- Add weighting when hardware differs across replicas
- Use dynamic batching with Triton for latency-bounded throughput
- Layer caching: templates, embeddings, tool results
- Reject IP hash unless session affinity is concretely beneficial



Kubernetes HPA in Practice

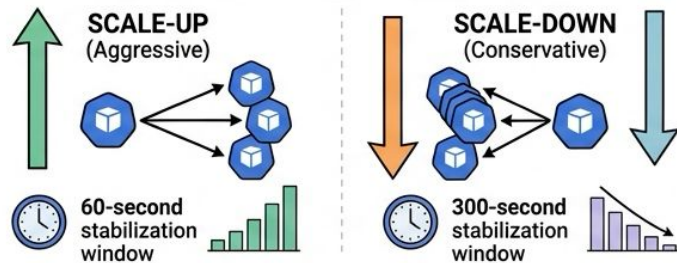
1. DEFINE RESOURCE REQUESTS AND LIMITS PER CONTAINER



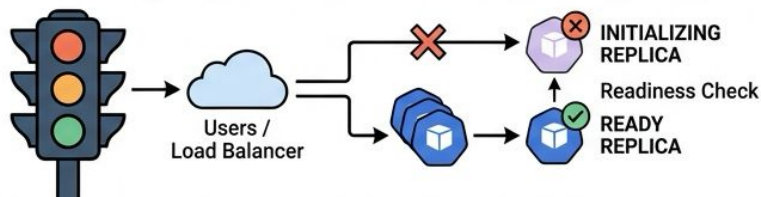
2. SET DUAL-METRIC TARGETS FOR SCALING



3. ASYMMETRIC SCALING BEHAVIOR



READINESS PROBES: GATE TRAFFIC UNTIL TRULY READY



Key Takeaways

- Scaling gives agents capacity; cognition gives them capability
 - Part 5 shifts from "how fast" to "how well agents think"
 - Chapter 5.1 introduces chain-of-thought reasoning
 - Planning and memory systems build on the infrastructure here
 - Infrastructure without intelligence is just expensive plumbing
- 